

# API Change Justification

It has been a topic of discussion between the sponsor and software team as to whether or not API changes are justifiable given the cost. This is a breakdown, from an engineering perspective, as to why this is the ideal time to make these API changes.

## Pros

### Compliance with NY State Law - Security

Security is an important part of software engineering. Like architecture, it's not something that is patched in later, it's integrated at the lowest level and needs to be thought of throughout the entire software development life cycle. The industry standard security policy for server to device communication is mutual TLS. It's an encryption policy that makes sure that both the device is a valid client, and that our API is a valid server. mTLS allows us to make and set security policies, allowlists, blocklists, quarantines, and other security policies that were previously impossible. As of right now, anyone with an exposed API Key (exposed as it was publicly available for over 24 hours) can access our API.

Additionally, we intend to propose this application for NY State use via the DEC. NYS has [regulations](#) for information security that apply to 3rd parties, such as minimum security requirements. Encryption of data in transit is one of those minimum requirements. Additionally, regulations require security to be integrated into the system throughout the entire software development life cycle. It wasn't before us, but we can't change that.

### Long Term Cost

Securing the system is a requirement, and there is no **better** time to do this than when there is only a single deployed device, rather than one device at all the trail heads. A transition can be planned for the future to update our device whenever is most convenient. Until the old server has been destroyed, data can be collected.

### Scalability

We have already started running into performance issues with the data model that we preemptively solved. As it stands, the firmware for V1 sends the device data at midnight every night if connected over cellular modem. There are 500+ trails in the Adirondacks. As we start to scale our operations and add more devices to the network, it's of paramount importance that we can handle at least 10 devices sending their trail data over the network to the API at the same time. With the current interface, this would mean transmitting potentially thousands of timestamps at a time. With the new interface, it would mean 10 devices would send only 10 separate fields at the same time, instead of thousands.

## Solid Foundations

We already have to make changes to any V1 device we make. If we have to make changes, we might as well do it in such a way that we can lay the foundation for something better.

## Nothing is Breaking - Until You Want It To

The API and persistence is tied to your terraform apply. You've applied the old website, and it will persist until you destroy it. Nothing is "broken" until we decide to transition to the new API interface.

## Cons

### Breaking Changes

Security changes to the device are breaking changes, and will require the V1 code to be changed. This would require making changes in `modem.c`. The API interface would be a trivial change compared to the security changes - it's in essence just a count of the timestamps rather than each individual timestamp.

### Device Inaccessibility

The biggest problem with this is how inaccessible the device is currently. If it were closer, this might not be such a problem. Unfortunately, there isn't really much we can do about this - but you also have to consider that the old API interface is going to be a bottleneck and much earlier than we can estimate, exactly like the data model. This is the easiest it will ever be to make changes to our API - we should take advantage of that to make sure that there will be no expensive problems in the future.